Practical Aspects of Deep Learning: Setting up ML Applications

1. The Iterative Nature of Applied ML

- **Challenge:** It is nearly impossible to correctly guess the optimal **hyperparameters** (number of layers, hidden units, learning rates, activation functions) on the first attempt.
- **Domain Specificity:** Intuitions from one domain (e.g., NLP) often do not transfer effectively to others (e.g., Computer Vision or Speech Recognition).
- **The Cycle:** Applied ML is a highly iterative process governed by the following loop:

1. **Idea:** Propose a model architecture and configuration.
2. **Code:** Implement the model.
3. **Experiment:** Run the model to evaluate performance.

- **Goal:** The efficiency of this cycle determines how quickly you find a high-performance network. Efficient data setup accelerates this process.

2. Dataset Splitting: Train, Dev, and Test Sets

Properly splitting data allows for efficient model selection and performance evaluation.

Definitions

- **Training Set:** The subset of data used to train the learning algorithm.
- **Dev Set (Development Set / Hold-out Cross Validation Set):** The subset used to evaluate different models and select the best performing one.
- **Test Set:** The subset used for a final, **unbiased estimate** of the selected model's performance.

Impact of Dataset Size on Split Ratios

The "best practice" ratios for splitting data have evolved with the advent of Big Data.

- **Traditional Machine Learning (Small Data)**
- *Context:* 100 to 10,000 examples.
- *Standard Ratios:*

- **70/30**: 70% Train, 30% Test.
- **60/20/20**: 60% Train, 20% Dev, 20% Test.
- *Reasoning:* These ratios were necessary to ensure significant statistical representation in small datasets.
- **Modern Deep Learning (Big Data)**
- *Context:* 1,000,000+ examples.
- *Strategy:* The Dev and Test sets only need to be large enough to distinguish between algorithms or provide a confident performance estimate. They do not need to be a large percentage of the total data.
- *Example:* If you have 1,000,000 examples, 10,000 is sufficient for Dev and 10,000 for Test.
- *Modern Ratios:*
- **98 / 1 / 1**: 98% Train, 1% Dev, 1% Test.
- **99.5 / 0.25 / 0.25**: For extremely large datasets.

3. Data Mismatch and Distribution Guidelines

In modern applications, training data often comes from a different distribution than the data used in production.

- **Scenario:**
- **Training Data:** High-resolution, professional images scraped from the web (plentiful).
- **Dev/Test Data:** Blurry, amateur images uploaded by mobile users (scarce, but represents the actual use case).
- **The Rule of Thumb:** Ensure that your **Dev Set and Test Set come from the same distribution**.
- Ideally, they should reflect the data you expect to encounter in the real application.
- It is acceptable for the Training Set to come from a different distribution if it allows for a larger dataset.

4. Absence of a Test Set

- **Usage:** It is acceptable to omit the Test Set if you do not require an unbiased estimate of the final system's performance.
- **Workflow:** Train on Training Set Iterate/Optimize on Dev Set.
- **Terminology Warning:** In scenarios with no Test Set, many teams refer to the Dev Set as the "Test Set."

- *Correction:* If a set is used to make decisions regarding model tuning, it is functionally a **Dev Set**, even if labeled otherwise.
- *Risk:* This leads to overfitting on the "Test Set" because it is part of the feedback loop.

Bias and Variance

1. Conceptual Overview

- **Importance:** Understanding bias and variance is a fundamental skill for ML practitioners. It is "easy to learn but difficult to master."
- **The "Trade-off":** In the modern Deep Learning era, the traditional "Bias-Variance Trade-off" is discussed less. Modern tools often allow reducing bias without hurting variance, and vice versa.
- **Visual Intuition (2D Data):**
- **High Bias (Underfitting):** A model that is too simple (e.g., a straight line) and fails to capture the underlying pattern of the data.
- **High Variance (Overfitting):** A model that is too complex (e.g., a high-degree polynomial) and fits the noise/outliers in the training data, failing to generalize.
- **"Just Right":** A model with intermediate complexity that captures the true pattern without fitting noise.

2. Diagnosing Bias and Variance

In high-dimensional problems, we cannot visualize the decision boundary. Instead, we diagnose issues by comparing **Training Set Error** and **Dev Set (Development) Error**.

Assumption: For the following diagnostics, we assume **Bayes Error (Optimal Error)** is nearly **0%** (e.g., a human can classify the data perfectly).

Diagnostic Scenarios

Training Set Error	Dev Set Error	Diagnosis	Interpretation
1% (Low)	**11%** (High)	**High Variance**	The model fits the training data well but fails to generalize to new data (**Overfitting**).

Training Set Error	Dev Set Error	Diagnosis	Interpretation
15% (High)	**16%** (High)	**High Bias**	The model fails to fit even the training data well (**Underfitting**). The generalization gap is small, but performance is poor overall.
15% (High)	**30%** (Very High)	**High Bias & High Variance**	The model underfits the data generally (high bias) but also overfits to specific outliers or noise (high variance). Worst-case scenario.
0.5% (Low)	**1%** (Low)	**Low Bias & Low Variance**	The ideal state. The model fits the training data well and generalizes effectively.

3. Key Nuances

- **Bayes Error Context:** The diagnosis changes if the problem is inherently difficult (e.g., blurry images where even humans have 15% error).
- *Example:* If Bayes Error is 15%, a **15% Training Error** represents **Low Bias**, not High Bias.
- *Rule:* Bias is assessed by comparing Training Error to the Optimal (Bayes) Error. Variance is assessed by comparing Dev Error to Training Error.
- **High Bias & High Variance Visualized:** Imagine a linear classifier (causing High Bias/Underfitting) that strangely curves to fit a few specific outliers (causing High Variance/Overfitting). This can occur in high-dimensional spaces.

Basic Recipe for Machine Learning

1. The Systematic Process

Once you have trained an initial model, use the following systematic "recipe" to improve performance based on the diagnostics (Bias vs. Variance) established in the previous section.

Step 1: Diagnose and Fix High Bias

- **Diagnostic:** Evaluate performance on the **Training Set**.
- **Goal:** Fit the training set well (achieve low training error).
- **Solutions for High Bias:**

1. **Bigger Network:** Increase the number of hidden layers or hidden units (almost always helps).
2. **Train Longer:** Increase the number of iterations/epochs.
3. **Advanced Optimization Algorithms:** (Covered later in the course).
4. **New Architecture:** Try a different neural network structure (e.g., CNN, RNN). *Note: This is less guaranteed to work than simply increasing scale.*

Step 2: Diagnose and Fix High Variance

- **Diagnostic:** Evaluate performance on the **Dev (Development) Set**.
- **Goal:** Generalize well to new data (minimize the gap between Training error and Dev error).
- **Solutions for High Variance:**

1. **More Data:** Acquiring more training data is the most reliable fix.
2. **Regularization:** Techniques to reduce overfitting (e.g., L2 regularization, Dropout).
3. **New Architecture:** Find an architecture better suited to the data structure.

*Repeat this cycle until the model achieves both **Low Bias** and **Low Variance**.*

2. The Evolution of the "Bias-Variance Trade-off"

- **Pre-Deep Learning Era:**
- Practitioners faced a strict **Trade-off**.
- Tools that reduced bias often increased variance, and vice versa.
- **Modern Deep Learning Era:**
- We now have tools to address each property independently.
- **Reducing Bias:** Training a **bigger network** almost always reduces bias without hurting variance (provided the network is properly regularized).
- **Reducing Variance:** Getting **more data** almost always reduces variance without hurting bias.
- **Key Takeaway:** In Deep Learning, you rarely have to balance the two carefully. You can often drive both down simultaneously given enough data and computational power.
- **Cost:** The primary downside to training strictly larger networks is increased **computational time**.
